



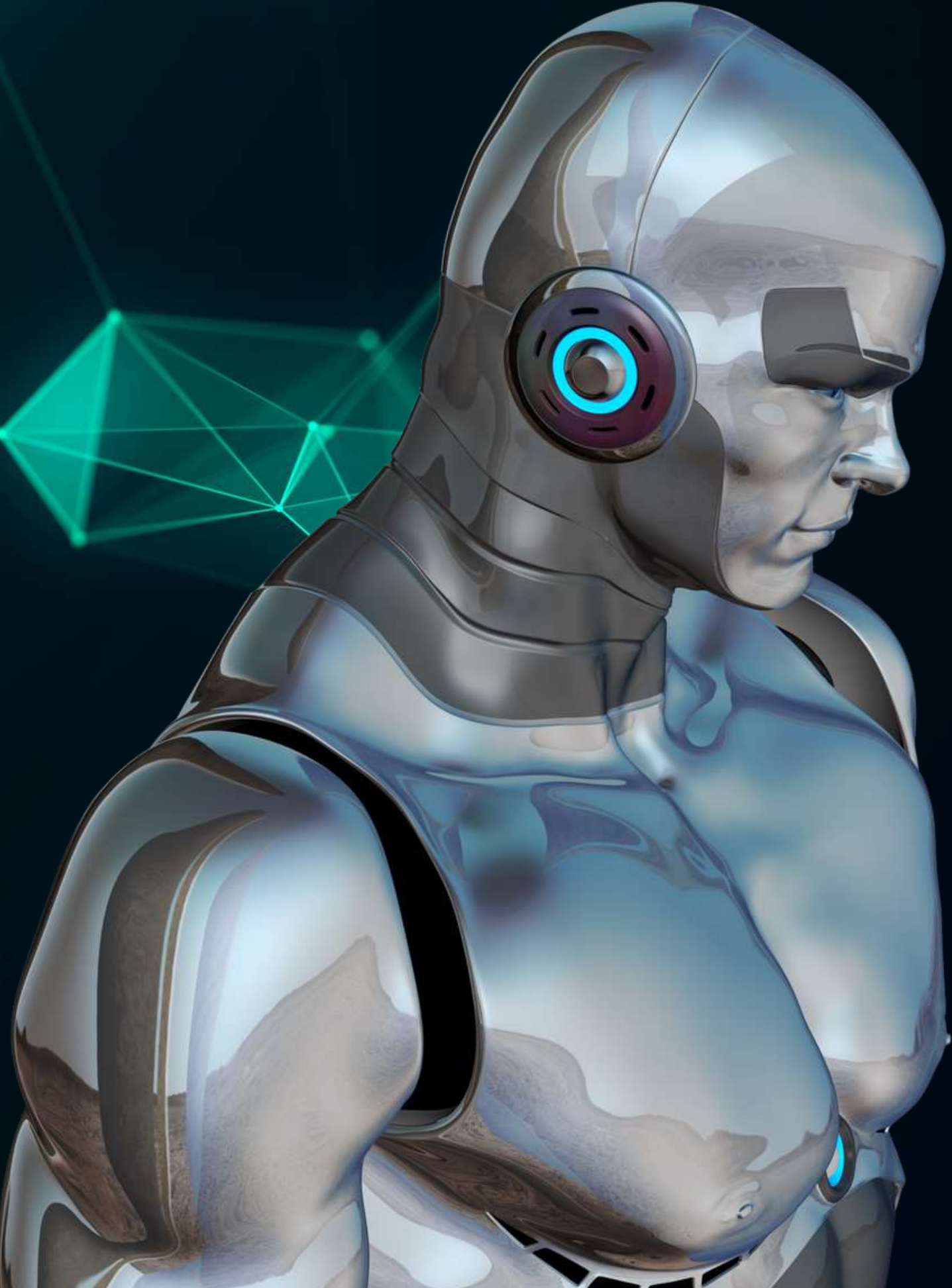
TAKSHASHILA UNIVERSITY

NAME: GOKUL .P

ROLL NO: TU6253210211015

SUBJECT CODE: U25CL07B

SUBJECT NAME: OBJECT ORIENTED PROGRAMMING IN JAVA



INTRODUCTION TO EXCEPTION HANDLING IN JAVA :

Meaning of Exception Handling :

- An exception is an unexpected event that occurs during program execution and disrupts the normal flow of instructions.
- Exception handling is a mechanism in Java that allows programmers to detect, handle, and recover from runtime errors without crashing the program.

Examples of exceptions:

- *Dividing a number by zero*
- *Accessing an invalid array index*
- *Reading a file that does not exist*



WHY EXCEPTION HANDLING IS IMPORTANT ?

- Prevents program crashes
- Maintains normal program flow
- Helps in debugging
- Improves program reliability

TYPES OF EXCEPTIONS IN JAVA :

Checked Exceptions :

- Checked at compile time.
- Must be handled using try-catch or throws.
- Example: IOException, SQLException.

Unchecked Exceptions :

- Occur at runtime.
- Not checked at compile time.
- Example: Arithmetic Exception, Null Pointer Exception, Array Index Out Of Bounds Exception.



KEYWORDS USED IN EXCEPTION HANDLING :

try :

- Used to define a block of code where an exception may occur.

catch:

- Used to handle the exception.

finally:

- Executes whether an exception occurs or not.

throw:

- Used to explicitly throw an exception.

throws:

- Declares exceptions that a method may throw.

BASIC SYNTAX :

```
try {  
    // code that may cause exception  
}  
catch(Exception e) {  
    // code to handle exception  
}  
finally {  
    // code that always executes  
}
```

EXAMPLE PROGRAM :

```
PUBLIC CLASS EXCEPTIONEXAMPLE {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        TRY {  
            INT A = 10;  
            INT B = 0;  
            INT RESULT = A / B;  
            SYSTEM.OUT.PRINTLN(RESULT);  
        }  
        CATCH (ARITHMETICEXCEPTION E) {  
            SYSTEM.OUT.PRINTLN("ERROR: CANNOT DIVIDE BY ZERO");  
        }  
        FINALLY {  
            SYSTEM.OUT.PRINTLN("PROGRAM FINISHED");  
        }  
    }  
}
```

OUTPUT:

```
ERROR: CANNOT DIVIDE BY ZERO  
PROGRAM FINISHED
```

CONCLUSION :

Exception Handling in Java HELPS MANAGE RUNTIME ERRORS EFFECTIVELY. BY USING TRY, CATCH, AND FINALLY, DEVELOPERS CAN PREVENT PROGRAM CRASHES AND ENSURE SMOOTHER EXECUTION OF APPLICATIONS.